ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

ĐỒ ÁN 1

THIẾT KẾ PHẦN CỨNG TĂNG TỐC AUTOENCODER KHỬ NHIỄU ẢNH TRÊN FPGA

Hardware Acceleration of Denoising Autoencoder and Handwritten Digit Recognition on FPGA

Giảng viên hướng dẫn: TS. Phạm Hoài Luân

Sinh viên thực hiện:

Nguyễn Minh Tuấn - 23521721
Cao Đăng Tùng - 23521737

Tp. Hồ Chí Minh, 2026

MỤC LỤC

Chương 1. GIỚI THIỆU ĐỀ TÀI 5

1.1 Lý do chọn đề tài 5

1.2 Mục tiêu đề tài: 5

Chương 2. CƠ SỞ LÝ THUYẾT 6

2.1. Mạng Nơ-ron Tích chập (CNN) 6

2.1.1. Phép toán tích chập (Convolution) 6

2.1.2. Hàm kích hoạt 6

2.1.3. Pooling Layer 7

2.1.4. Fully Connected Layer 7

2.1.5. Dropout Layer 7

2.1.6. Kiến trúc lenet cổ điển 7

2.2. Tổng quan về công nghệ FPGA 8

2.2.1. Kiến trúc cơ bản của FPGA 8

2.2.2. Ưu điểm của FPGA trong triển khai mạng nơ-ron 9

2.2.3. Quy trình thiết kế với FPGA 9

2.2.4. Công cụ chuyển đổi Py2C 9

2.3. High-Level Synthesis - HLS 10

2.3.1. Khái niệm về HLS 10

2.3.2. Kiểu dữ liệu Số định điểm 10

2.3.3. Giao tiếp AXI và tích hợp hệ thống 11

2.4. Tập dữ liệu chữ số viết tay MNIST 11

2.4.1. Giới thiệu chung 11

2.4.2. Đặc điểm kỹ thuật 11
2.4.3. Quy trình tiền xử lý trong đồ án 12
2.4.4. Tầm quan trọng đối với thực nghiệm FPGA 12
Chương 3. TRIỂN KHAI ĐỀ TÀI 12
3.1. Thiết kế Mô hình 12
3.1.1. Kiến trúc mạng đề xuất 12
3.1.2. Triển khai và Huấn luyện Mô hình 14
3.1.3. Chuyển đổi mô hình từ Python sang C/C++ 16
3.2 Code C++ và tối ưu 19
3.2.1. Phân tích mã nguồn sinh ra từ công cụ Py2C 19
3.2.2. Chiến lược chỉnh sửa và tối ưu hóa mã C++ 19
3.2.3 Kết quả báo cáo Synthesis 20
3.3 Tích hợp IP vào hệ thống SoC 22
3.3.1 Kiến trúc 22
3.3.2 Address mapping 23
Chương 4. KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ 25
4.1 Dữ liệu kiểm thử 25
4.2 Hiện thực phần mềm điều khiển hệ thống 27
4.3 Kết quả thực nghiệm 28
4.4 Đánh giá kết quả 29
Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 29
5.1. Kết luận 30
5.2. Hướng phát triển 30

DANH MỤC HÌNH ẢNH

Hình 1 Các file được tạo ra sau khi chạy Py2C 17
Hình 2 Kết quả sau khi chuyển từ Python sang C++ 18
Hình 3 Report từ Vitis HLS 18
Hình 4 Kết quả báo cáo Synthesis 20
Hình 5 SoC hoàn chỉnh 22
Hình 6 Address map 24
Hình 7 Testcase được lưu trong file header 26
Hình 8 Lưu đồ điều khiển 27
Hình 9 Inference loop 28
Hình 10 Kết quả sau khi chạy 28
Hình 11 Bảng tài nguyên của bài báo Accelerator Implementation of Lenet-5 Convolution Neural Network Based on FPGA with HLS 29

TÓM TẮT ĐỒ ÁN

Đồ án trình bày quy trình thiết kế và hiện thực hóa kiến trúc mạng nơ-ron tích chập CNN cổ điển LeNet-5 trên nền tảng FPGA sử dụng phương pháp tổng hợp mức cao High-Level Synthesis - HLS. Với mục tiêu giải quyết bài toán nhận dạng chữ số viết tay trên tập dữ liệu MNIST, đồ án chuyển đổi thuật toán từ ngôn ngữ C/C++ sang ngôn ngữ mô tả phần cứng thông qua công cụ Vivado Vitis HLS. Các kỹ thuật tối ưu hóa kiến trúc song song như Pipelining, Loop Unrolling được áp dụng triệt để nhằm giảm độ trễ và tăng thông lượng xử lý. Khối IP Core sau khi tổng hợp được tích hợp vào hệ thống trên chip và giao tiếp qua chuẩn AXI. Kết quả thực nghiệm cho thấy hệ thống đạt độ chính xác 98.7%.

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1 Lý do chọn đề tài

- Trong những năm gần đây, Trí tuệ nhân tạo, đặc biệt là Deep Learning, đã đạt được những thành tựu đột phá trong lĩnh vực thị giác máy tính. Tuy nhiên, việc triển khai các mạng nơ-ron tích chập (CNN) trên các nền tảng phần cứng truyền thống thường gặp thách thức về tiêu thụ năng lượng và độ trễ thời gian thực.

- Nền tảng FPGA nổi lên như một giải pháp tối ưu nhờ khả năng tính toán song song mạnh mẽ và tính linh hoạt cao. Tuy nhiên, việc thiết kế phần cứng bằng ngôn ngữ mô tả phần cứng truyền thống đòi hỏi nhiều thời gian và công sức. Do đó, phương pháp Tổng hợp mức cao (High-Level Synthesis - HLS) ra đời, cho phép chuyển đổi từ các ngôn ngữ bậc cao như C/C++ sang RTL, giúp rút ngắn chu kỳ phát triển sản phẩm mà vẫn đảm bảo hiệu năng. Đó là lý do đề tài "Thiết kế và hiện thực hóa kiến trúc LeNet-5 trên FPGA bằng phương pháp HLS" được lựa chọn.

1.2 Mục tiêu đề tài:

- Xây dựng cấu trúc mạng LeNet-5 tối ưu cho việc nhận dạng chữ số viết tay từ bộ dữ liệu MNIST.

- Sử dụng công cụ Vitis HLS để chuyển đổi thuật toán từ C/C++ sang khối IP Core phần cứng.
- Áp dụng các kỹ thuật tối ưu hóa phần cứng Pipelining, Loop Unrolling để tối ưu độ trễ và tài nguyên sử dụng trên FPGA.
- Tích hợp hệ thống trên chip SoC thông qua chuẩn giao tiếp AXI và kiểm chứng độ chính xác trên phần cứng thực tế.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Mạng Nơ-ron Tích chập (CNN)

Mạng nơ-ron tích chập là một loại mạng nơ-ron học sâu (Deep Learning) được thiết kế đặc biệt để xử lý các dữ liệu có cấu trúc lưới như hình ảnh **[1].** Thay vì kết nối mọi nơ-ron ở tầng trước với mọi nơ-ron ở tầng sau (như mạng MLP truyền thống), CNN sử dụng phép toán tích chập để nhận diện các đặc trưng cục bộ.

2.1.1. Phép toán tích chập (Convolution)

Đây là thành phần cốt lõi của mạng. Một cửa sổ nhỏ gọi là **Kernel** sẽ trượt qua toàn bộ hình ảnh đầu vào. Tại mỗi vị trí, hệ thống thực hiện phép nhân chập giữa các giá trị của Filter với giá trị pixel tương ứng để tạo ra một **Feature Map**.

Công thức toán học của phép tích chập 2D:

$$G[m,n] = (f * h)[m,n] = \sum_j \sum_k h[j,k] f[m-j, n-k]$$ **[2]**

Trong đó:

- f: Hình ảnh đầu vào.
- h: Ma trận Filter.
- G: Kết quả đầu ra.

2.1.2. Hàm kích hoạt

Sau mỗi phép tính tích chập, một hàm phi tuyến tính được áp dụng để mạng có thể học được các mẫu phức tạp.

- ReLU: Phổ biến nhất hiện nay vì tính toán nhanh, công thức $f(x) = \max(0, x)$. **[12]**

- Sigmoid/Tanh: Thường dùng trong kiến trúc LeNet-5 cổ điển để nén giá trị đầu ra vào khoảng (0, 1) hoặc (-1, 1). **[1]**

2.1.3. Pooling Layer

Tầng này có nhiệm vụ giảm kích thước không gian của Feature Map, giúp giảm số lượng tham số cần tính toán và tránh hiện tượng overfitting. **[13]**

- Max Pooling: Lấy giá trị lớn nhất trong cửa sổ lọc.
- Average Pooling: Lấy giá trị trung bình.

2.1.4. Fully Connected Layer

Sau khi qua nhiều tầng tích chập và pooling, dữ liệu được Flatten thành một vector một chiều. Vector này đi qua các tầng FC để thực hiện bài toán phân loại cuối cùng

2.1.5. Dropout Layer

Tầng Dropout: Đây là kỹ thuật điều chỉnh .Bằng cách ngắt ngẫu nhiên các nơ ron trong quá trình huấn luyện, mạng sẽ không bị phụ thuộc quá mức vào một nhóm nơ-ron nhất định, từ đó tránh hiện tượng overfitting và đạt độ chính xác cao trên tập dữ liệu mới.**[14]**

2.1.6. Kiến trúc lenet cổ điển [1]

Dựa trên yêu cầu tối ưu hóa tài nguyên phần cứng trên nền tảng FPGA nhưng vẫn đảm bảo độ chính xác cao (98.6%), kiến trúc mạng được triển khai trong đồ án có các thông số cấu hình cụ thể như sau:

- **Tầng đầu vào (Input Layer):** Tiếp nhận ảnh xám từ bộ dữ liệu MNIST có kích thước 28x28 pixel. Dữ liệu được chuẩn hóa về khoảng [0, 1].
- **Tầng C1 (Convolutional Layer):** Sử dụng **4 bộ lọc** kích thước 5x5. Kết quả tạo ra 4 bản đồ đặc trưng (Feature Maps) kích thước 24x24
- **Tầng S2 (Max Pooling Layer):** Thực hiện lấy giá xtrị lớn nhất trong vùng cửa sổ 2x2 với bước nhảy (stride) bằng 2. Kích thước bản đồ đặc trưng giảm xuống còn 12x12
- **Tầng C3 (Convolutional Layer):** Sử dụng **8 bộ lọc** kích thước 5x5. Kết quả tạo ra 8 bản đồ đặc trưng kích thước 8x8.

- **Tầng S4 (Max Pooling Layer):** Tiếp tục thực hiện Max pooling 2x2, giảm kích thước dữ liệu xuống còn 4x4.
- **Tầng phẳng hóa (Flatten):** Chuyển đổi dữ liệu từ dạng ma trận 4x4x8 sang vector một chiều gồm 128 phần tử để đưa vào các tầng kết nối đầy đủ.
- **Tầng F5 (Fully Connected Layer):** Gồm **32 nơ-ron** với hàm kích hoạt ReLU. Đây là nơi kết hợp các đặc trưng bậc cao đã trích xuất được.
- **Tầng F6 (Fully Connected Layer):** Gồm **16 nơ-ron** với hàm kích hoạt ReLU, giúp tinh lọc thông tin trước khi phân loại.
- **Tầng đầu ra (Output Layer):** Gồm **10 nơ-ron** sử dụng hàm kích hoạt **Softmax**, tương ứng với xác suất dự đoán của 10 chữ số từ 0 đến 9.

2.2. Tổng quan về công nghệ FPGA

2.2.1. Kiến trúc cơ bản của FPGA

FPGA (Field Programmable Gate Array) là một loại mạch tích hợp cỡ lớn mà người dùng có thể lập trình cấu hình logic sau khi sản xuất **[3]**. Khác với CPU hay GPU có cấu trúc phần cứng cố định, FPGA cho phép người thiết kế xây dựng các cấu trúc phần cứng chuyên biệt phù hợp với từng thuật toán cụ thể.

Các thành phần chính bên trong một chip FPGA bao gồm:

- **Configurable Logic Blocks:** Các khối logic có thể cấu hình, chứa các LUT và Flip-flops để thực hiện các hàm logic phức tạp.
- **DSP Slices:** Các bộ tính toán số học chuyên dụng, thực hiện cực nhanh phép toán nhân-cộng. Đây là thành phần quan trọng nhất để tăng tốc tầng Convolution trong CNN.
- **Block RAM:** Các khối bộ nhớ nội trong chip, cho phép truy xuất dữ liệu với băng thông cực lớn và độ trễ thấp, dùng để lưu trữ trọng số và dữ liệu trung gian của mạng nơ-ron.
- **Hệ thống kết nối:** Các kênh dây dẫn linh hoạt cho phép kết nối các thành phần trên lại với nhau theo ý muốn của người thiết kế.

2.2.2. Ưu điểm của FPGA trong triển khai mạng nơ-ron

Việc hiện thực hóa kiến trúc LeNet-5 trên FPGA mang lại nhiều lợi thế so với kiến trúc von Neumann :

1. **Tính song song cao:** FPGA có thể thực hiện hàng ngàn phép tính nhân chập cùng một lúc bằng cách phân phối dữ liệu trên hàng trăm bộ DSP, giúp giảm đáng kể thời gian xử lý ảnh.
2. **Khả năng tùy biến kiến trúc:** Người thiết kế có thể tinh chỉnh chính xác độ rộng của dữ liệu để tiết kiệm tài nguyên mà không làm giảm nhiều độ chính xác.
3. **Hiệu năng trên mỗi Watt:** FPGA tiêu thụ ít năng lượng hơn nhiều so với GPU trong khi vẫn duy trì được tốc độ xử lý cao.
4. **Cấu trúc Pipeline:** FPGA cho phép thiết kế các đường ống dữ liệu, giúp dữ liệu vào và ra được xử lý liên tục, tối ưu hóa thông lượng của hệ thống.

2.2.3. Quy trình thiết kế với FPGA

Trong đồ án này, thay vì sử dụng ngôn ngữ mô tả phần cứng truyền thống vốn đòi hỏi thời gian thiết kế dài, quy trình được tối ưu hóa thông qua High-Level Synthesis (HLS). HLS đóng vai trò là cầu nối, tự động chuyển đổi các mô tả thuật toán mức cao thành các kiến trúc phần cứng tối ưu, cho phép người thiết kế tập trung vào việc tối ưu thuật toán CNN thay vì lo lắng về các kết nối logic chi tiết ở mức cổng.

2.2.4. Công cụ chuyển đổi Py2C

Do sự khác biệt về ngôn ngữ giữa khâu huấn luyện (Python) và khâu thiết kế phần cứng (C++), đồ án sử dụng công cụ Py2C [10]. Công cụ này có thể tự động phân tích tệp mô hình đã huấn luyện (.h5) và sinh ra các tệp mã nguồn C++ (.cpp, .h). Đồng thời nó giúp đóng gói trọng số và cấu trúc mạng, đồng thời hỗ trợ cấu hình định dạng số định điểm để chuẩn bị cho quá trình HLS. Chi tiết cách dùng công cụ này nằm ở tài liệu tham khảo..

2.3. High-Level Synthesis - HLS

2.3.1. Khái niệm về HLS

High-Level Synthesis (HLS) là một quy trình thiết kế điện tử tự động hóa việc chuyển đổi các mô tả thuật toán ở mức trừu tượng cao (thường là ngôn ngữ C, C++ hoặc SystemC) sang ngôn ngữ mô tả phần cứng như Verilog hoặc VHDL.

Trong đồ án này, công cụ **Vitis HLS** được sử dụng để tổng hợp cấu trúc mạng LeNet-5. Thay vì phải thiết kế từng khối nhân, bộ cộng và thanh ghi ở mức cổng, HLS cho phép người thiết kế tập trung vào kiến trúc hệ thống và sử dụng các chỉ thị pragmas để điều khiển cách phần cứng được tạo ra.

2.3.2. Kiểu dữ liệu Số định điểm

Một trong những khác biệt lớn nhất giữa phần mềm máy tính và phần cứng FPGA là cách biểu diễn số liệu. Trong khi Python hay CPU thường sử dụng số thực dấu phẩy động như Floating-point 32-bit/64-bit để đạt độ chính xác cao, thì loại dữ liệu này lại tiêu tốn rất nhiều tài nguyên logic và gây độ trễ lớn khi triển khai trên FPGA.

Để giải quyết vấn đề này, HLS cung cấp thư viện số định điểm ap_fixed **[6].** Số định điểm sử dụng cơ chế tương tự số nguyên để biểu diễn các giá trị thập phân, bằng cách cố định vị trí của dấu chấm thập phân. Định dạng tổng quát được khai báo là ap_fixed<W, I>, trong đó:

W (Word length): Tổng số bit của biến.

I (Integer length): Số bit dành cho phần nguyên (bao gồm cả bit dấu).

Số bit còn lại $(W - I)$ được dành cho phần thập phân.

Việc sử dụng số định điểm giúp giảm đáng kể tài nguyên tính toán (đặc biệt là các khối DSP và LUT) và tăng tốc độ xử lý, trong khi vẫn duy trì được độ chính xác ở mức chấp nhận được so với mô hình gốc.

2.3.3. Giao tiếp AXI và tích hợp hệ thống

Để bộ tăng tốc LeNet-5 hoạt động được trên hệ thống SoC, nó cần giao tiếp với bộ vi xử lý và bộ nhớ ngoài. Chuẩn giao tiếp AMBA AXI được sử dụng để chuẩn hóa các kết nối này: **[7]**

AXI4-Lite được sử dụng cho kênh điều khiển. CPU đóng vai trò là Master, gửi các lệnh khởi động, cấu hình tham số và đọc trạng thái từ khối LeNet-5.

AXI4-Stream được sử dụng cho kênh truyền dữ liệu tốc độ cao. Dữ liệu ảnh đầu vào và trọng số được truyền trực tiếp từ bộ nhớ vào FPGA để xử lý mà không cần địa chỉ phức tạp cho từng pixel, thường kết hợp với bộ điều khiển DMA) để giảm tải cho CPU.

Sau khi tổng hợp thành công IP Core, khối LeNet-5 được xuất sang Vivado Design Suite. Tại đây, IP được kết nối với hệ thống xử lý thông qua các Interconnect trên sơ đồ khối để tạo thành file bitstream hoàn chỉnh nạp xuống FPGA.

2.4. Tập dữ liệu chữ số viết tay MNIST

2.4.1. Giới thiệu chung

MNIST (Modified National Institute of Standards and Technology) là bộ dữ liệu chuẩn mực nhất **[8]** trong lĩnh vực Machine Learning và Computer Vision, được sử dụng rộng rãi để đánh giá hiệu năng của các hệ thống nhận dạng hình ảnh. Bộ dữ liệu này được xây dựng bởi Yann LeCun, Corinna Cortes và Christopher Burges thông qua việc xử lý lại bộ dữ liệu gốc từ NIST.

2.4.2. Đặc điểm kỹ thuật

Tập dữ liệu MNIST bao gồm các đặc điểm chính sau:

- **Tổng số mẫu:** 70.000 mẫu chữ số viết tay từ 0 đến 9.
    - **Tập huấn luyện** gồm 60.000 mẫu, dùng để dạy mạng nơ-ron nhận diện các nét viết.
    - **Tập kiểm tra** gồm 10.000 mẫu, dùng để đánh giá khách quan độ chính xác của mạng sau khi huấn luyện.
- Mỗi mẫu là một ảnh xám có kích thước 28x28 pixel.
- Mỗi pixel có giá trị cường độ sáng nằm trong khoảng từ 0 đến 255.

2.4.3. Quy trình tiền xử lý trong đồ án

Để phù hợp với kiến trúc mạng CNN triển khai trên FPGA bằng HLS, dữ liệu MNIST trong đồ án này được xử lý qua các bước:

1. Chuyển đổi giá trị pixel từ khoảng nguyên [0, 255] sang khoảng số thực [0, 1] bằng cách chia cho 255. Việc này giúp các trọng số của mạng không bị gradient explosion và hội tụ nhanh hơn.
2. Ảnh đầu vào được đưa về dạng ma trận 28x28x1 để thực hiện phép toán tích chập 2D.
3. Các nhãn từ 0 đến 9 được chuyển đổi sang dạng One-hot Encoding. Điều này phục vụ cho việc tính toán hàm mất mát ở tầng đầu ra Softmax.

2.4.4. Tầm quan trọng đối với thực nghiệm FPGA

Việc sử dụng MNIST giúp đồ án có một thước đo chuẩn mực. Độ chính xác đạt được trong quá trình huấn luyện phần mềm sẽ là mốc tham chiếu để so sánh với kết quả thực thi trên phần cứng FPGA sau khi đã thực hiện tối ưu hóa và chuyển đổi định dạng dữ liệu sang Fixed-point.

Chương 3. TRIỂN KHAI ĐỀ TÀI

3.1. Thiết kế Mô hình

Trước khi tiến hành thiết kế phần cứng trên FPGA, việc xây dựng một mô hình phần mềm hoạt động chính xác là bước tiên quyết. Mô hình này đóng vai trò để tham chiếu, kiểm chứng độ chính xác và trích xuất các tham số weights/bias cần thiết cho quá trình tổng hợp mức cao.

3.1.1. Kiến trúc mạng đề xuất

Nhằm tối ưu hóa hiệu năng tính toán đồng thời tiết kiệm tài nguyên tài nguyên (DSP, BRAM) vốn rất hạn chế trên FPGA, đồ án đã thực hiện cải tiến kiến trúc LeNet-5 cổ điển thành một phiên bản tinh gọn. Thay vì sử dụng hàng chục bộ lọc như nguyên bản, đồ án tập trung vào việc giảm độ phức tạp nhưng vẫn duy trì khả năng trích xuất đặc trưng mạnh mẽ.

Các cải tiến cụ thể trong kiến trúc của đồ án bao gồm:

- **Tối ưu hóa số lượng Filter.** Chỉ sử dụng 4 bộ lọc ở lớp C1 và 8 bộ lọc ở lớp C3. Việc cắt giảm này giúp giảm hơn 10 lần số lượng tham số so với LeNet-5 gốc, giúp IP Core sau khi tổng hợp có diện tích nhỏ, dễ dàng tích hợp vào các dòng FPGA giá rẻ.
- **Sử dụng kỹ thuật Dropout.** Bổ sung hai lớp Dropout (tỉ lệ 0.2 và 0.3) sau các lớp kết nối đầy đủ. Kỹ thuật này giúp mạng tăng tính tổng quát hóa, chống lại hiện tượng quá khớp (overfitting), đảm bảo mô hình hoạt động ổn định trên các nét chữ viết tay thực tế.
- **Hàm kích hoạt và Pooling.** Sử dụng hàm **ReLU** để tăng tốc độ hội tụ và **Max Pooling** để giữ lại các đặc trưng nổi bật nhất của chữ số, thay cho Average Pooling truyền thống.
- **Phân loại.** Mô hình được thiết kế để phân loại 10 lớp trạng thái tương ứng với các chữ số từ 0 đến 9 trên bộ dữ liệu MNIST.

Bảng 3.1. Cấu trúc và số tham số của model Lenet gốc

Layer (type)	**Output Shape**	**Parameter**
Input	(32, 32, 1)	0
Conv1 (C1)	(28, 28, 6)	156
AvgPool (S2)	(14, 14, 6)	12
Conv2 (C3)	(10, 10, 16)	2416
AvgPool (S4)	(5, 5, 16)	32
Dense (C5)	-120	48120
Dense (F6)	-84	10164
Output	-10	850
TỔNG CỘNG		61750

Bảng 3.2. Cấu trúc và số tham số của model Lenet tham chiếu

Layer (type)	**Output Shape**	**Parameter**
Input	(28, 28, 1)	0

Conv1 (C1)	(24, 24, 4)	104
MaxPool (S2)	(12, 12, 4)	0
Conv2 (C3)	(8, 8, 8)	808
MaxPool (S4)	(4, 4, 8)	0
Flatten	(128)	0
Dropout_1 (0.2)	(128)	0
Dense (C5)	(32)	4128
Dropout_2 (0.3)	(32)	0
Dense (F6)	(16)	528
Output	10	170
TỔNG CỘNG		**5738**

3.1.2. Triển khai và Huấn luyện Mô hình

Mô hình được triển khai bằng ngôn ngữ lập trình Python với thư viện TensorFlow/Keras – một framework phổ biến và mạnh mẽ cho học sâu. Việc sử dụng Keras Sequential API giúp xây dựng mạng một cách ngắn gọn, dễ đọc và dễ dàng điều chỉnh cấu trúc.

3.1.2.1. Chuẩn bị dữ liệu

Bộ dữ liệu MNIST được tải trực tiếp từ Keras datasets, gồm:

- 60.000 ảnh huấn luyện và 10.000 ảnh kiểm tra.
- Mỗi ảnh có kích thước 28x28 pixel, grayscale (1 channel).
- Nhãn là các chữ số từ 0 đến 9.

Các bước tiền xử lý:

- Chuẩn hóa giá trị pixel từ [0, 255] về [0, 1] để tăng tốc độ hội tụ và ổn định huấn luyện.
- Thêm chiều channel: reshape từ (N, 28, 28) thành (N, 28, 28, 1).
- Chuyển nhãn sang dạng one-hot encoding (ví dụ: nhãn 5 → [0,0,0,0,0,1,0,0,0,0]).

Không áp dụng data augmentation trong phiên bản này để giữ tính đơn giản và tập trung vào hiệu suất cơ bản, nhưng có thể bổ sung sau nếu cần cải thiện độ tổng quát hóa.

3.1.2.2. Quy trình huấn luyện

Quá trình huấn luyện mô hình được thực hiện trên môi trường Python nhằm tìm ra bộ tham số Weights và Bias tối ưu nhất cho mạng CNN trước khi triển khai lên phần cứng. Các thiết lập cụ thể như sau:

- Sử dụng **Adam** với tốc độ học mặc định là 0.001. Adam được lựa chọn vì khả năng tự động điều chỉnh tốc độ học cho từng tham số, giúp mô hình hội tụ nhanh và ổn định hơn so với thuật toán SGD truyền thống.
- Sử dụng hàm mất mát **Categorical Cross-Entropy**. Đây là hàm mục tiêu tối ưu cho bài toán phân loại đa lớp, giúp cực đại hóa xác suất của lớp đúng thông qua việc tính toán sai lệch giữa phân phối dự đoán và nhãn thực tế.
- **Chiến lược huấn luyện:**
    - **Batch size:** 64. Batch size vừa phải giúp cân bằng giữa tốc độ tính toán và độ chính xác của gradient.
    - **Epochs:** 40. Qua thực nghiệm, tại epoch thứ 40, mô hình đạt trạng thái hội tụ tốt nhất mà không xảy ra hiện tượng bão hòa sớm.
    - **Phân chia dữ liệu:**10% từ 60,000 mẫu huấn luyện, 6,000 mẫu được tách ra làm tập kiểm định để theo dõi hiệu năng của mô hình sau mỗi epoch, đảm bảo tính khách quan trước khi đánh giá trên tập Test.
- Nhằm đảm bảo mô hình hoạt động tốt trên các dữ liệu chữ viết tay mới, đồ án áp dụng hai lớp **Dropout** với tỉ lệ 20% và 30% sau các lớp Dense. Kỹ thuật này ngắt ngẫu nhiên một số kết nối, buộc mạng nơ-ron phải học các đặc trưng mang tính tổng quát thay vì ghi nhớ máy móc dữ liệu huấn luyện để tránh overfitting.

3.1.2.4. Kết quả huấn luyện

Sau 40 epochs, mô hình đạt kết quả cao trên tập kiểm tra:

- Test loss: ~0.04
- Test accuracy: 98.6%

Bảng 3.3. Kết quả huấn luyện

Epoch	Train Loss	Train Accuracy	Val Loss	Val Accuracy
1	0.512	84.2%	0.312	90.8%
5	0.112	96.5%	0.098	97.1%
10	0.078	97.8%	0.065	98.0%
20	0.055	98.4%	0.048	98.3%
40	0.042	98.7%	0.041	98.4%

3.1.2.4. So sánh với LeNet gốc

Kết quả thực nghiệm trên phần mềm cho thấy mô hình LeNet mới đã tối ưu hóa vượt trội về mặt cấu trúc. Cụ thể, số lượng tham số đã giảm khoảng 10.7 lần so với kiến trúc LeNet-5 nguyên bản (từ hơn 60,000 xuống còn 5,738 tham số). Mặc dù cấu trúc được tinh gọn đáng kể để phù hợp với tài nguyên hạn chế của FPGA, mô hình vẫn duy trì được hiệu năng cao với độ chính xác đạt 98.6% trên tập dữ liệu kiểm tra. Điều này chứng minh tính hiệu quả của việc cải tiến kiến trúc trong việc cân bằng giữa chi phí phần cứng và chất lượng nhận dạng.

3.1.3. Chuyển đổi mô hình từ Python sang C/C++

Để triển khai mô hình đã huấn luyện trên Python (Keras) lên nền tảng FPGA thông qua công nghệ HLS, mã nguồn cần được chuyển đổi sang ngôn ngữ C/C++. Quá trình này giải quyết hai thách thức cốt lõi:

- Chuyển từ các lớp trừu tượng trong Keras sang các vòng lặp và logic xử lý tường minh trong C++.
- Chuyển đổi từ số Floating-point mặc định của Python sang số Fixed-point trên phần cứng để tiết kiệm tài nguyên và năng lượng cho FPGA.

3.1.3.1 Công cụ chuyển đổi tự động Py2C

Đồ án sử dụng công cụ Py2C để tự động hóa việc sinh mã C++. Với fixed point được lựa chọn là Q16.16 với 16 bit là phần nguyên và 16 bit làm phần thập phân

Hình 1 Các file được tạo ra sau khi chạy Py2C

3.1.3.2 Kiểm chứng tính đúng đắn khi chuyển đổi mô hình sang C/C++

Để kiểm chứng được tính đúng đắn của code C/C++ sau khi được sinh ra từ công cụ Py2C thì đồ án có sử dụng cách chạy mô phỏng C Simulation trên phần mềm Vitis HLS. Với 50 testcase là 50 hình đầu tiên của tập dữ liệu MINST

```
Result: 7
Result: 2
Result: 7
Result: 1
Result: 2
Result: 1
Result: 1
Result: 7
Result: 4
Result: 2
Result: 3
Result: 5
Result: 1
Result: 2
Result: 4
Result: 4
accuracy of Model: 100%
INFO: [SIM 1] CSim done with 0 errors.
INFO: [SIM 3] *************** CSIM finish ***************
```

Hình 2 Kết quả sau khi chuyển từ Python sang C++

Có thể thấy code C/C++ đã được chuyển đổi đúng logic khi accuracy xấp xỉ lúc huấn luyện mô hình.

3.1.3.3 Báo cáo tài nguyên của Py2C

Sau khi kiểm tra tính đúng đắn của mô hình bằng C-Simulation, đồ án tiến hành quá trình **Tổng hợp mức cao (HLS)** để đánh giá các thông số về tài nguyên phần cứng và thời gian thực thi. Hình 3 là kết quả báo cáo từ Vitis HLS đối với mã nguồn được sinh tự động bởi Py2C.

Performance & Resource Estimates

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
CNN				-	155684	1.557E6	-	155685	-	no	40	242	20997	25004	0
CNN_Pipeline_loop_for_channel2D_0_loop_for_bp2D_0_loop_for_ap2D_0	II Violation			-	80652	8.070E5	-	80652	-	no	0	75	1816	3656	0
CNN_Pipeline_VITIS_LOOP_7_1_VITIS_LOOP_9_2_VITIS_LOOP_10_3				-	579	5.790E3	-	579	-	no	0	0	90	727	0
CNN_Pipeline_VITIS_LOOP_30_1_VITIS_LOOP_32_2_VITIS_LOOP_33_3				-	131	1.310E3	-	131	-	no	0	0	81	546	0
CNN_Pipeline_VITIS_LOOP_51_1_VITIS_LOOP_52_2_VITIS_LOOP_53_3				-	130	1.300E3	-	130	-	no	0	0	54	353	0
CNN_Pipeline_loop_for_a_Dense_0_loop_for_b_Dense_0	II Violation			-	8202	8.202E4	-	8202	-	no	0	3	696	758	0
CNN_Pipeline_VITIS_LOOP_14_1				-	34	340.000	-	34	-	no	0	0	15	92	0
CNN_Pipeline_loop_for_a_Dense_1	II Violation			-	562	5.620E3	-	562	-	no	0	96	5272	5193	0
CNN_Pipeline_VITIS_LOOP_29_1				-	18	180.000	-	18	-	no	0	0	13	91	0
Dense_2	II Violation			-	319	3.190E3	-	319	-	no	5	64	9240	8275	0
loop_for_channel2D_1_loop_for_bp2D_1_loop_for_ap2D_1				-	65024	6.500E5	127	-	512	no	-	-	-	-	-

Performance Pragma

Hình 3 Report từ Vitis HLS

Nhận xét về BRAM: Việc sử dụng tới 40 khối BRAM chủ yếu do Py2C khai báo các mảng đệm (Buffer) trung gian dưới dạng các khối nhớ độc lập cho từng tầng. Điều này tạo ra sự lãng phí khi dữ liệu không được tái sử dụng hiệu quả.

Nhận xét về DSP: Chỉ số DSP cao (242 bộ) do các vòng lặp chưa được áp dụng các chỉ thị tối ưu hóa, khiến công cụ tổng hợp tự động tạo ra nhiều bộ nhân số thực riêng lẻ cho mỗi phép tính tích chập.

3.2 Code C++ và tối ưu

3.2.1. Phân tích mã nguồn sinh ra từ công cụ Py2C

Mã nguồn C++ được sinh ra tự động từ công cụ Py2C đóng vai trò là khung sườn cơ bản. Tuy nhiên, qua quá trình kiểm tra trong Vitis HLS, mã nguồn này bộc lộ một số hạn chế về mặt phần cứng:

- **Cấu trúc rời rạc:** Mã gen tự động thường tách các tầng thành các khối hàm độc lập với các buffer riêng biệt. Điều này dẫn đến việc sử dụng lãng phí tài nguyên BRAM để lưu trữ dữ liệu trung gian.
- Việc sử dụng các hàm toán học tiêu chuẩn và các vòng lặp chưa được tối ưu khiến công cụ tổng hợp tạo ra quá nhiều bộ nhân-cộng rời rạc, gây bùng nổ tài nguyên DSP.
- Mã mặc định thường thực hiện truy cập mảng một cách tuần tự, không tận dụng được băng thông của FPGA, dẫn đến độ trễ cao.

3.2.2. Chiến lược chỉnh sửa và tối ưu hóa mã C++

Để khắc phục các nhược điểm trên và phù hợp với mô hình 4-filter/8-filter, đồ án đã tiến hành tái cấu trúc lại mã nguồn theo các hướng sau:

Thay vì các vòng lặp rời rạc, đồ án sử dụng các kỹ thuật:

- Thay vì duy trì các hàm riêng biệt cho từng tầng (Convolution, Pooling, Dense) như mã nguồn gốc từ Py2C, đồ án đã tái cấu trúc lại thành một hàm tính toán duy nhất với các tham số đầu vào linh hoạt. Việc gộp các chức năng này mang lại những lợi ích quan trọng là **Tối ưu hóa tài nguyên phần cứng, Tiết kiệm bộ nhớ BRAM, Đơn giản hóa điều khiển:**
    - Thay vì yêu cầu FPGA khởi tạo nhiều khối logic riêng lẻ cho từng tầng, việc gộp hàm cho phép **tái sử dụng** các bộ nhân-cộng và bộ logic. Khi thực hiện xong tầng Convolution, các bộ tài nguyên này ngay lập tức

được cấu hình lại để xử lý tầng Pooling hoặc Dense, giúp giảm đáng kể tổng diện tích chip sử dụng.

  - Bằng cách sử dụng chung một hàm, hệ thống có thể quản lý các buffer tập trung hơn. Thay vì mỗi hàm có một mảng nội bộ riêng, các tầng sẽ chia sẻ các vùng nhớ trung gian theo cơ chế gối đầu, giúp giảm số lượng khối BRAM tiêu tốn.
  - Việc quản lý luồng dữ liệu dataflow trở nên đơn giản hơn khi bộ vi xử lý chỉ cần điều phối các tham số đầu vào để quyết định chức năng của khối tính toán tại thời điểm đó.

- **Pipeline:** Áp dụng #pragma HLS PIPELINE cho các vòng lặp tính tích chập. Điều này cho phép thực hiện phép nhân-cộng cho pixel tiếp theo ngay khi pixel hiện tại đang được xử lý ở công đoạn sau, giúp tối ưu hóa hệ số sử dụng tài nguyên và giảm chu kỳ máy trên mỗi vòng lặp (Initiation Interval - II).
- • **Inline:** Áp dụng `#pragma HLS INLINE` cho các hàm con tính toán hoặc các khối logic bổ trợ. Chỉ thị này yêu cầu công cụ tổng hợp phẳng hóa cấu trúc hàm bằng cách chèn trực tiếp mã nguồn vào vị trí gọi hàm. Việc này giúp loại bỏ độ trễ do thao tác gọi hàm và cho phép các tối ưu hóa khác có thể xuyên suốt qua các ranh giới hàm, từ đó tối ưu hóa logic tổng thể tốt hơn.
- **Unroll:** Áp dụng `#pragma HLS UNROLL` cho các vòng lặp duyệt qua các filter. Thay vì thực hiện tuần tự từng bộ lọc trên một bộ nhân duy nhất, chỉ thị này yêu cầu FPGA khởi tạo nhiều bộ DSP song song. Điều này cho phép tính toán đồng thời kết quả của nhiều filter trong cùng một chu kỳ máy, giúp tăng tốc độ xử lý lên gấp nhiều lần dựa trên tính song song đặc trưng của phần cứng.

3.2.3 Kết quả báo cáo Synthesis

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
CNN				-	139021	1.390E6	-	139022	-	no	14	116	7195	14478	0
Conv2D	II Violation			-	59914	5.990E5	-	59914	-	no	0	100	2032	4069	0
Conv2D_6				-	5761	5.761E4	-	5761	-	no	0	0	137	866	0
Conv2D_5				-	62465	6.250E5	-	62465	-	no	0	4	485	1611	0
Conv2D_4				-	1153	1.153E4	-	1153	-	no	0	0	105	514	0
CNN_Pipeline_VITIS_LOOP_49_1_VITIS_LOOP_51_2_VITIS_LOOP_53_3				-	130	1.300E3	-	130	-	no	0	0	54	353	0
Conv2D_3	II Violation			-	8202	8.202E4	-	8202	-	no	0	4	611	930	0
Conv2D_2	II Violation			-	1034	1.034E4	-	1034	-	no	0	4	596	897	0
Conv2D_1	II Violation			-	347	3.470E3	-	347	-	no	0	4	719	1156	0

Hình 4 Kết quả báo cáo Synthesis

Sau khi tiến hành tái cấu trúc mã nguồn C++ và áp dụng các chỉ thị tối ưu hóa phần cứng, đồ án đã tiến hành chạy Synthesis để đánh giá sự thay đổi về tài nguyên.

A. So sánh tài nguyên trước và sau tối ưu hóa

Kết quả cho thấy việc chỉnh sửa mã nguồn thủ công đã mang lại sự cải thiện vượt trội về mặt diện tích chip (Area) so với mã nguồn sinh tự động mặc định.

Bảng 3.4: Bảng so sánh tài nguyên giữ mô hình trước khi tối ưu và sau khi tối ưu

Tài nguyên	Trước tối ưu	Sau tối ưu	Tỉ lệ giảm (%)	Trạng thái
BRAM	40	14	65.0%	Tối ưu tốt
DSP	242	116	52.1%	Tối ưu tốt
FF	20,097	7,195	64.2%	Tối ưu tốt
LUT	25,004	14,478	42.1%	Tối ưu tốt

B. Phân tích nguyên nhân cải thiện

Sự sụt giảm mạnh mẽ các chỉ số tài nguyên được giải thích bởi các lý do kỹ thuật sau:

1. **Giảm BRAM (40 → 14)** nhờ việc gộp các tầng tính toán vào một hàm duy nhất và tái sử dụng các buffer, hệ thống không còn phải khởi tạo nhiều khối bộ nhớ riêng biệt cho mỗi lớp, giúp tiết kiệm hơn 65% dung lượng BRAM.
2. **Giảm DSP (242 → 116)** nhờ việc sử dụng hàm tính toán chung giúp bộ tổng hợp HLS nhận diện và thực hiện chia sẻ phần cứng. Các bộ DSP được tái sử dụng qua các giai đoạn khác nhau của mạng nơ-ron thay vì khởi tạo dư thừa cho từng tầng riêng lẻ.
3. **Giảm FF và LUT:** Kỹ thuật HLS INLINE đã làm phẳng cấu trúc logic, loại bỏ các bộ điều khiển hàm rườm rà. Bên cạnh đó, việc chuyển đổi sang kiểu dữ liệu fixed-point giúp giảm đáng kể số lượng cổng logic cần thiết để thực hiện các phép toán so với floating-point.

C. Đánh giá về Timing

Mặc dù tài nguyên giảm mạnh, nhưng nhờ vào chỉ thị PIPELINE và UNROLL, tốc độ thực thi vẫn được đảm bảo, thậm chí còn ổn định hơn nhờ giảm bớt các thao tác truy xuất bộ nhớ không cần thiết. Hệ thống hoàn toàn đáp ứng mức xung nhịp **100MHz** với độ an toàn về thời gian Slack cao.

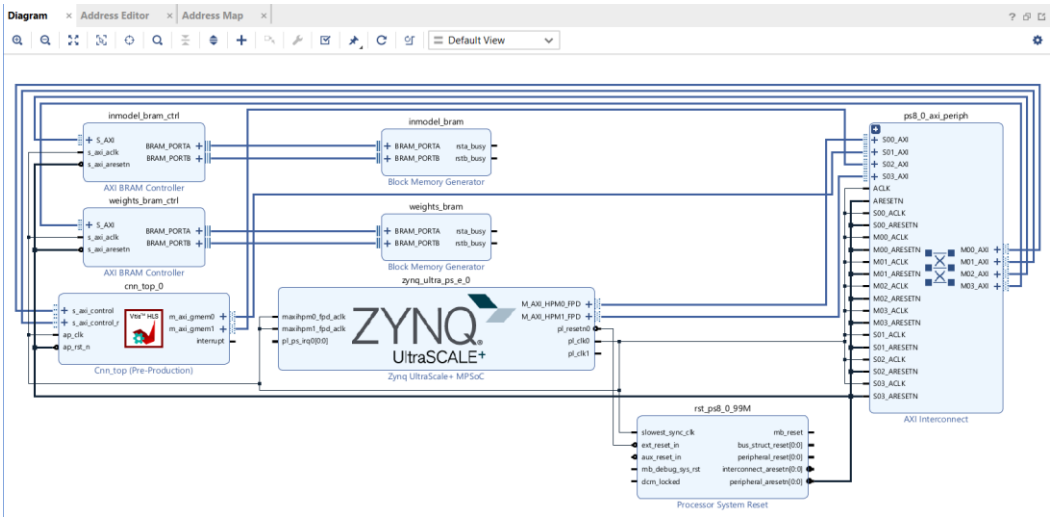
Kết luận:

Kết quả báo cáo Synthesis khẳng định mô hình hiện tại đã đạt tới trạng thái tối ưu, sẵn sàng để tích hợp vào hệ thống SoC trên FPGA mà vẫn còn dư thừa nhiều tài nguyên cho các tính năng mở rộng khác.

3.3 Tích hợp IP vào hệ thống SoC

3.3.1 Kiến trúc

Hệ thống nhận diện chữ số viết tay được xây dựng dựa trên nền tảng Zynq UltraScale+ MPSoC. Thiết kế tận dụng sức mạnh xử lý điều khiển của khối Processing System và khả năng tính toán song song tốc độ cao của khối Programmable Logic .



Hình 5 SoC hoàn chỉnh

Sơ đồ khối tổng thể của hệ thống được thể hiện trong hình trên, bao gồm các thành phần chính sau:

1. **Zynq UltraScale+ MPSoC** đóng vai trò là khối điều khiển trung tâm. Khối này chịu trách nhiệm:
   - Khởi tạo hệ thống và cấu hình các ngoại vi.
   - Nạp dữ liệu ảnh đầu vào và bộ trọng số vào bộ nhớ.
   - Gửi tín hiệu điều khiển xuống khối gia tốc phần cứng.
   - Thu thập kết quả phân lớp sau khi quá trình tính toán hoàn tất.
2. **Khối Cnn_top_0** là nhân IP Core được tổng hợp từ mã nguồn C++ thông qua công cụ Vitis HLS. Khối này sở hữu hai giao diện giao tiếp chính:

- Giao diện điều khiển (s_axi_control) sử dụng chuẩn AXI4-Lite, cho phép CPU ghi vào các thanh ghi điều khiển và đọc trạng thái hoạt động.
- Giao diện truy cập bộ nhớ (m_axi_gmem) sử dụng chuẩn AXI4 Master, cho phép khối CNN chủ động truy xuất dữ liệu từ bộ nhớ đệm mà không cần sự can thiệp liên tục của CPU, giúp giảm tải cho vi xử lý.

3. **Hệ thống bộ nhớ đệm** để tối ưu hóa tốc độ truy xuất và giảm độ trễ so với việc dùng RAM ngoài, hệ thống sử dụng kiến trúc bộ nhớ chia sẻ dựa trên Block RAM:
   - AXI BRAM Controller đóng vai trò cầu nối, chuyển đổi giao thức AXI thành tín hiệu điều khiển đọc/ghi vật lý cho BRAM.
   - Block Memory Generator là nơi lưu trữ thực tế cho dữ liệu mô hình. Việc tách riêng bộ nhớ cho đầu vào và trọng số giúp tăng băng thông truy cập dữ liệu song song.
4. **AXI Interconnect** đóng vai trò là bộ định tuyến trung tâm, quản lý luồng dữ liệu giữa Master và các Slave.

3.3.2 Address mapping

Dựa trên công cụ Address Editor trong Vivado, sơ đồ phân hoạch bộ nhớ được thiết lập cụ thể như sau:

Diagram | Address Editor | Address Map

Assigned (12) | Unassigned (0) | Excluded (4) | Hide All

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/cnn_top_0					
/cnn_top_0/Data_m_axi_gmem0 (64 address bits : 16E)					
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A002_0000	8K	0x0000_0000_A002_1FFF
/weights_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A002_8000	32K	0x0000_0000_A002_FFFF
Excluded (2)					
/cnn_top_0/Data_m_axi_gmem1 (64 address bits : 16E)					
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A002_0000	8K	0x0000_0000_A002_1FFF
/weights_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A002_8000	32K	0x0000_0000_A002_FFFF
Excluded (2)					
/zynq_ultra_ps_e_0					
/zynq_ultra_ps_e_0/Data (40 address bits : 0x00A0000000 [256M] ,0x0400000000 [4G] ,0x1000000000 [224G] ,0x00B0000000 [256M] ,0x0500000000 [4G] ,0x4800000000 [22					
/cnn_top_0/s_axi_control	s_axi_control	Reg	0x00_A000_0000	64K	0x00_A000_FFFF
/cnn_top_0/s_axi_control	s_axi_control	Reg	0x00_A000_0000	64K	0x00_A000_FFFF
/cnn_top_0/s_axi_control_r	s_axi_control_r	Reg	0x00_A001_0000	64K	0x00_A001_FFFF
/cnn_top_0/s_axi_control_r	s_axi_control_r	Reg	0x00_A001_0000	64K	0x00_A001_FFFF
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A002_0000	8K	0x00_A002_1FFF
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A002_0000	8K	0x00_A002_1FFF
/weights_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A002_8000	32K	0x00_A002_FFFF
/weights_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A002_8000	32K	0x00_A002_FFFF

Hình 6 Address map

Trong đó:

1. **Vùng nhớ Điều khiển:**

Đây là vùng địa chỉ mà CPU (Zynq UltraScale+) sử dụng để gửi lệnh và cấu hình cho IP Core.

- **cnn_top_0/s_axi_control** có địa chỉ cơ sở **0x00_A000_0000** (Range 64K). Đây là nơi chứa các thanh ghi trạng thái như ap_start, ap_done, ap_idle.
- **cnn_top_0/s_axi_control_r**: Địa chỉ cơ sở **0x00_A001_0000** (Range 64K). Đây là giao tiếp mở rộng được thiết kế để chứa các thanh ghi tham số đầu vào. Chương trình trên CPU sẽ đọc địa chỉ cơ sở của BRAM và ghi giá trị này vào thanh ghi tương ứng bên trong s_axi_control_r.

2. **Vùng nhớ Dữ liệu:**

Đây là vùng lưu trữ chung nơi chứa dữ liệu mô hình. Cả CPU và IP Core đều nhìn thấy vùng nhớ này tại cùng một địa chỉ vật lý, đảm bảo tính toàn vẹn dữ liệu.

- **inmodel_bram_ctrl**: Địa chỉ cơ sở **0x00_A002_0000** với kích thước **8K**. Vùng này được dành riêng để lưu trữ ảnh đầu vào.
- **weights_bram_ctrl**: Địa chỉ cơ sở **0x00_A002_8000** với kích thước **32K**. Vùng này lớn hơn, được dành riêng để lưu trữ bộ trọng số và Bias của mạng LeNet-5.

3. **Cơ chế Đa truy cập (Multi-port Access):**

Khối IP Core cnn_top_0 sở hữu hai giao diện master riêng biệt là **Data_m_axi_gmem0** và **Data_m_axi_gmem1**.

- Việc tách biệt này cho phép IP Core truy cập song song vào inmodel_bram và weights_bram cùng một lúc. Điều này tối ưu hóa băng thông bộ nhớ: trong khi một kênh đang đọc dữ liệu pixel ảnh, kênh còn lại có thể nạp trọng số cho lớp tiếp theo mà không gây nghẽn, giúp cải thiện tốc độ xử lý của hệ thống.

Chương 4. KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ

4.1 Dữ liệu kiểm thử

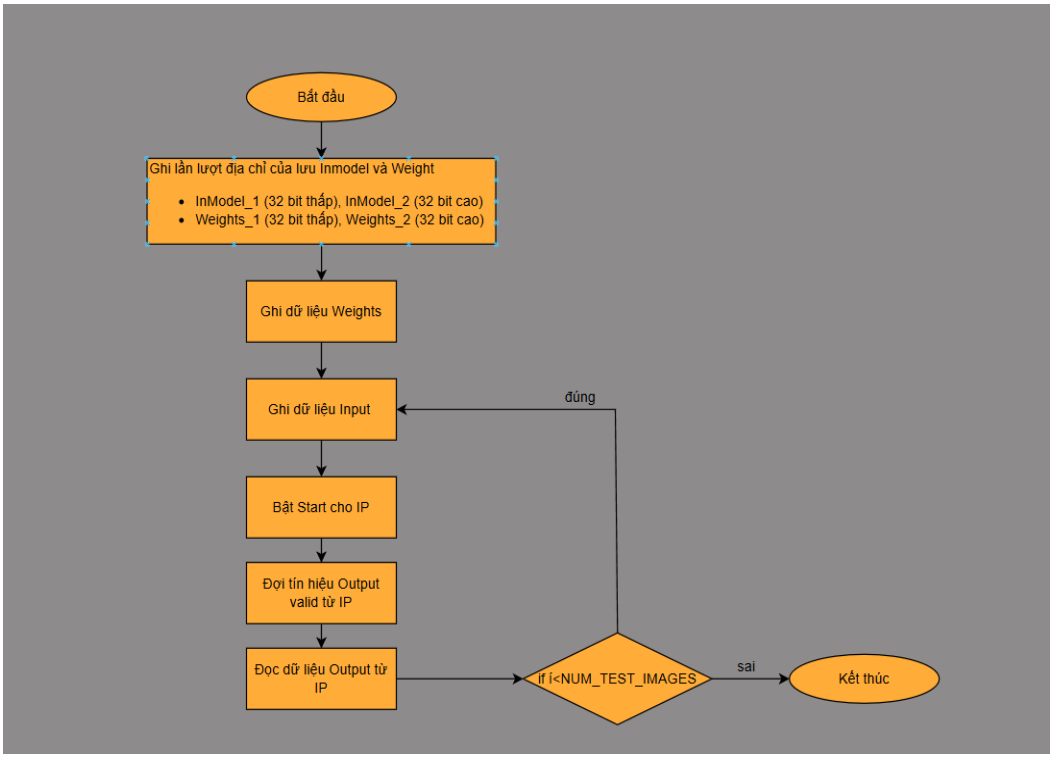
Để đánh giá độ chính xác của hệ thống, nhóm đã chuẩn bị tập dữ liệu gồm 1.000 ảnh từ bộ MNIST Test Set. Do phần cứng được thiết kế tối ưu hóa, toàn bộ dữ liệu pixel của ảnh đã được chuyển đổi từ số thực sang định dạng Fixed-point Q16.16. Dữ liệu này được đóng gói thành các mảng hằng số và lưu trong file header, cho phép dễ dàng truy xuất và nạp xuống bộ nhớ BRAM để thực hiện suy luận

```
const uint8_t test_labels[NUM_TEST_IMAGES] = {
    0, 7, 9, 4, 4, 8, 5, 5, 4, 0, 8, 2, 1, 0, 8, 4,
    5, 0, 4, 0, 6, 1, 7, 3, 2, 6, 7, 2, 6, 9, 3, 1,
    4, 6, 2, 5, 4, 2, 0, 6, 2, 1, 7, 3, 4, 1, 0, 5,
    4, 3, 1, 1, 7, 4, 9, 9, 4, 8, 4, 0, 2, 4, 5, 1,
    1, 6, 4, 7, 1, 9, 4, 2, 4, 1, 5, 5, 3, 8, 3, 1,
    4, 5, 6, 8, 9, 4, 1, 5, 3, 8, 0, 3, 2, 5, 1, 2,
    8, 3, 4, 4, 0, 8, 8, 3, 3, 1, 7, 3, 5, 9, 6, 3,
    2, 6, 1, 3, 6, 0, 7, 2, 1, 7, 1, 4, 2, 4, 2, 1,
    7, 9, 6, 1, 1, 2, 4, 8, 1, 7, 7, 4, 8, 0, 7, 3,
    1, 3, 1, 0, 7, 7, 0, 3, 5, 5, 2, 7, 6, 6, 9, 2,
    8, 3, 5, 2, 2, 5, 6, 0, 8, 2, 9, 2, 8, 8, 8, 8,
    7, 4, 9, 3, 0, 6, 6, 3, 2, 1, 3, 2, 2, 9, 3, 0,
    0, 5, 7, 8, 1, 4, 4, 6, 0, 2, 9, 1, 4, 7, 4, 7,
    3, 9, 8, 8, 4, 7, 1, 2, 1, 2, 2, 3, 2, 3, 2, 3,
    9, 1, 7, 4, 0, 3, 5, 5, 8, 6, 3, 2, 6, 7, 6, 6,
    3, 2, 7, 8, 1, 1, 7, 5, 6, 4, 9, 5, 1, 3, 3, 4,
    7, 8, 9, 1, 1, 6, 9, 1, 4, 4, 5, 4, 0, 6, 2, 2,
    3, 1, 5, 1, 2, 0, 3, 8, 1, 2, 6, 7, 1, 6, 2, 3,
    9, 0, 1, 2, 2, 0, 8, 9};

// Fixed-point format: Q16.16
const int32_t test_images_fixed[NUM_TEST_IMAGES][PIXELS_PER_IMAGE] = {
    {  // Image 0
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
```

Hình 7 Testcase được lưu trong file header

4.2 Hiện thực phần mềm điều khiển hệ thống



Hình 8 Lưu đồ điều khiển

1. **Nạp trọng số**

CPU thực hiện đọc toàn bộ bộ trọng số và ghi xuống vùng nhớ BRAM.

2. **Vòng lặp Suy luận**

Hệ thống thực hiện lặp lại quy trình xử lý cho từng ảnh trong tập dữ liệu:

- CPU nạp dữ liệu pixel của ảnh hiện tại xuống vùng nhớ BRAM
- CPU ghi giá trị 1 vào thanh ghi điều khiển ap_start để ra lệnh cho phần cứng bắt đầu tính toán.
- CPU đi vào trạng thái chờ, liên tục kiểm tra tín hiệu Done từ IP Core. Quá trình này đảm bảo CPU chỉ đọc kết quả khi phần cứng đã xử lý xong.
- Khi tín hiệu Valid được bật, CPU đọc giá trị dự đoán từ thanh ghi kết quả của IP Core.

3. **Kiểm tra điều kiện lặp**

Sau khi xử lý xong một ảnh, hệ thống kiểm tra biến đếm số lượng ảnh ($i < NUM_TEST_IMAGES$).

Nếu đúng → Quay lại bước "Ghi dữ liệu Input" để xử lý ảnh tiếp theo.

Nếu sai → Kết thúc chương trình.

```c
    for (int i = 0; i < NUM_TEST_IMAGES; i++)
    {
        printf("Image %d: ", i);

        int predicted = run_inference(test_images_fixed[i]);
        int expected = test_labels[i];

        if (predicted == expected)
        {
            printf("PASS - Predicted: %d, Expected: %d ✓\n", predicted, expected);
            correct++;
        }
        else
        {
            printf("FAIL - Predicted: %d, Expected: %d ✗\n", predicted, expected);
        }
    }

    // Print summary
    printf("\n========================================\n");
    printf("  RESULTS\n");
    printf("========================================\n");
    printf("Total images: %d\n", NUM_TEST_IMAGES);
    printf("Correct:      %d\n", correct);
    printf("Accuracy:     %.1f%%\n", (float)correct / NUM_TEST_IMAGES * 100.0f);
    printf("========================================\n\n");
```

Hình 9 Inference loop

4.3 Kết quả thực nghiệm

```
Image 988: PASS - Predicted: 1, Expected: 1 ✓
Image 989: PASS - Predicted: 6, Expected: 6 ✓
Image 990: PASS - Predicted: 2, Expected: 2 ✓
Image 991: PASS - Predicted: 3, Expected: 3 ✓
Image 992: PASS - Predicted: 9, Expected: 9 ✓
Image 993: PASS - Predicted: 0, Expected: 0 ✓
Image 994: PASS - Predicted: 1, Expected: 1 ✓
Image 995: PASS - Predicted: 2, Expected: 2 ✓
Image 996: PASS - Predicted: 2, Expected: 2 ✓
Image 997: PASS - Predicted: 0, Expected: 0 ✓
Image 998: PASS - Predicted: 8, Expected: 8 ✓
Image 999: PASS - Predicted: 9, Expected: 9 ✓

========================================
  RESULTS
========================================
Total images: 1000
Correct:      987
Accuracy:     98.7%
========================================

Some tests failed.
debian@BeagleBone:~/lenet$
```

Hình 10 Kết quả sau khi chạy

Sau khi chạy kiểm thử trên tập 1000 ảnh, hệ thống đạt độ chính xác 98.7%

4.4 Đánh giá kết quả

TABLE II. FPGA RESOURCE UTILIZATION

Resource	*DSP*	*BRAM*	*LUT*	*FF*
Used	*125*	*119.5*	*14659*	*14172*
Available	*220*	*140*	*53200*	*106400*
Utilization	*56.82%*	*85.36%*	*27.55%*	*13.32%*

[11]

Hình 11 Bảng tài nguyên của bài báo Accelerator Implementation of Lenet-5 Convolution Neural Network Based on FPGA with HLS

Bảng 4.1 so sánh tài nguyên giữa mô hình của đồ án và bài báo:

	Mô hình Lenet trong đồ án	Mô hình của bài báo
DSP	116	125
BRAM	14	119.5
LUT	14478	14659
FF	7195	14172

Nhận xét:

Dựa trên bảng so sánh, mô hình của đồ án thể hiện sự vượt trội về khả năng tối ưu hóa tài nguyên phần cứng, cụ thể là **Tối ưu BRAM, FF & LUT, DSP**

1. Việc chỉ sử dụng 14 khối BRAM cho phép hệ thống triển khai dễ dàng trên các dòng FPGA giá rẻ có tài nguyên hạn chế.
2. Cấu trúc lại mã nguồn và sử dụng kiểu dữ liệu Fixed-point giúp đơn giản hóa các khối điều khiển và thanh ghi lưu trữ dữ liệu.
3. Mô hình đồ án sử dụng ít hơn 9 bộ DSP nhờ chia sẻ tài nguyên giữa các tầng Convolution và Dense.
4. **Tổng kết:** Mô hình của đồ án không chỉ đạt độ chính xác cao (98.7%) mà còn đạt được sự cân bằng tối ưu giữa hiệu năng và chi phí tài nguyên. Với lượng tài nguyên tiết kiệm được, chip FPGA có thể tích hợp thêm các module xử lý ảnh đầu vào hoặc giao tiếp ngoại vi khác, tăng tính ứng dụng thực tế cho hệ thống.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đồ án đã thiết kế và hiện thực thành công bộ tăng tốc phần cứng cho mạng nơ-ron tích chập LeNet-5 trên FPGA. Các kết quả đạt được bao gồm:

1. Ứng dụng thành công công nghệ High-Level Synthesis để chuyển đổi thuật toán từ C++ sang RTL, giúp rút ngắn đáng kể thời gian phát triển so với phương pháp HDL truyền thống.
2. Hệ thống sử dụng định Fixed-point Q16.16 và cơ chế lưu trữ BRAM, giúp tiết kiệm tài nguyên tính toán nhưng vẫn đảm bảo độ chính xác nhận diện cao trên tập dữ liệu MNIST (đạt 98,7%).
3. Xây dựng hoàn chỉnh hệ thống System-on-Chip với giao tiếp AXI

Kết quả thực nghiệm chứng minh tính khả thi và hiệu năng vượt trội của FPGA trong các ứng dụng AI nhúng, đáp ứng tốt các yêu cầu về tốc độ xử lý và độ trễ thấp.

5.2. Hướng phát triển

Dựa trên nền tảng của đồ án này, các hướng nghiên cứu và cải tiến tiếp theo bao gồm:

1. Chuyển đổi từ định dạng Q16.16 xuống INT8. Việc này sẽ giảm 4 lần dung lượng bộ nhớ và tăng tốc độ xử lý lên đáng kể mà ít ảnh hưởng đến độ chính xác.
2. Mở rộng bộ nhớ: Hiện tại hệ thống đang dùng BRAM. Hướng phát triển tiếp theo là sử dụng bộ nhớ ngoài DDR4 kết hợp với cơ chế DMA để có thể chạy các mô hình lớn hơn như VGG16, ResNet hoặc YOLO.
3. Áp dụng thêm các kỹ thuật Dataflow và Systolic Array trong HLS để khai thác tối đa khả năng xử lý song song của FPGA.

TÀI LIỆU THAM KHẢO

[1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition*. Proceedings of the IEEE, 86(11), 2278–2324.

[2] Zhang, Q., et al. (2019). *Recent Advances in Convolutional Neural Network Acceleration*. Neurocomputing, 323, 37–51.

[3] Nurvitadhi, E., et al. (2017). *Can FPGAs Beat GPUs in Accelerating Next-Generation Deep Neural Networks?*. Proceedings of FPGA '17, 5–14.

[4] Shawahna, A., et al. (2019). *FPGA-Based Accelerators of Deep Learning Networks: A Review*. IEEE Access, 7, 7823–7859.

[5] Ali, N., et al. (2022). *Generating Efficient FPGA-Based CNN Accelerators from High-Level Descriptions*. Journal of Signal Processing Systems.

[6] Solovyev, R., et al. (2018). *Fixed-Point Convolutional Neural Network for Real-Time Video Processing in FPGA*. arXiv:1808.09945.

[7] Xilinx (AMD). (2023). *AXI Basics 1 - Introduction to AXI*. Adaptive Support Documentation

[8] LeCun, Y., Cortes, C., & Burges, C. J. C. (1998). *The MNIST Database of Handwritten Digits*.

[9] Grother, P. J. (1995). *NIST Special Database 19: Handprinted Forms and Characters Database*. NIST.

[10] Wang, N. (2023). *Py2C_new: A Python to C++ Converter for HLS*. GitHub: github.com/WangNe2207/Py2C_new.

[11] D. Rongshi and T. Yongming, "Accelerator Implementation of Lenet-5 Convolution Neural Network Based on FPGA with HLS," in *2019 3rd International Conference on Circuits, System and Simulation (ICCSS)*, Nanjing, China, 2019, pp. 64-67.

[12] Nair, V., & Hinton, G. E. (2010). *Rectified linear units improve restricted boltzmann machines*. In Proceedings of the 27th international conference on machine learning (ICML-10).

[13] Boureau, Y. L., Ponce, J., & LeCun, Y. (2010). *A theoretical analysis of feature pooling in visual recognition*. In Proceedings of the 27th international conference on machine learning (ICML-10).

[14] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: a simple way to prevent neural networks from overfitting*. The journal of machine learning research.